

EVENTIFY

Technical Analysis, Architectural Design & Interview Guide

Prepared For: Kinaxis Associate Consultant - Technology Placement

Core Focus Area: Backend Development, Relational Mapping, Concurrency Control, and API Testing

Tech Stack Analyzed: Node.js, Express, MongoDB (Mongoose ODM), JWT, Bcrypt, and Jest

This technical document serves as a complete evaluation of the Eventify platform. It structures implementation details, architectural choices, and database methodologies to align directly with the core engineering competencies required for technology consulting roles at Kinaxis.

1. What the Project Is and How It Works

Eventify is a secure, multi-tier web application built to streamline campus event planning, discoverability, ticket booking, and check-in management. The platform operates on a MERN (MongoDB, Express, React, Node.js) architecture, establishing separate presentation, application, and database layers to maintain modularity and ease of extension.

The application divides users into two major categories: **Event Organizers** and **Event Attendees**. The system handles end-to-end event lifecycles through the following operational flow:

- **Secure Session Establishment:** Users register and log into the system. Passwords are securely hashed on the server. Successful authentication returns a cryptographic JSON Web Token (JWT) embedded in a secure, HTTP-only cookie. This ensures stateless session persistence across requests without exposing credentials to client-side scripts.
- **Event Creation & Image Pipeline:** Organizers schedule events by inputting details (title, capacity, price, venue, date, time) and uploading promotional banner images. The server intercepts incoming image streams via a binary parser middleware, compresses/writes files to static storage, and stores relative paths in the database.
- **Ticketing & Inventory Management:** Attendees browse available events and purchase tickets. When booking, the backend validates available inventory. Upon confirmation, the ticket details are serialized and saved, and the event capacity is decremented atomically.
- **Check-in and Dynamic Wallet:** Attendees access their booked tickets via a personal wallet screen. The client fetches tickets dynamically and calls a public QR code generation API, embedding the unique ticket database identifier. During check-in, organizers scan the QR code to validate entry against the backend database.

2. The Problem Eventify Resolves

Campus environments host hundreds of departmental, cultural, and sports events annually. However, they frequently suffer from operational friction caused by manual processes. Eventify addresses four major pain points:

1. **Fragmented Coordination:** Campuses typically rely on disjointed tools like social media groups, emails, and shared documents. Eventify centralizes event discovery and planning, integrating a scheduling database with a visual frontend calendar view.
2. **Vulnerable and Complex Sign-Ins:** Storing plain text or weakly hashed passwords makes databases goldmines for malicious actors. Furthermore, saving session keys in localStorage exposes sessions to Cross-Site Scripting (XSS) hijacking. Eventify solves this with adaptive salting/hashing and HTTP-only cookie parameters.
3. **Ticket Double-Booking (Race Conditions):** Traditional student apps suffer from concurrency bugs. If two users try to book the last available ticket simultaneously, simple database reads followed by writes can allow both bookings to succeed, overselling the event. Eventify uses database-level atomic operations to ensure inventory integrity.
4. **Audit Trail Corruption (Pricing Shifts):** If an organizer updates an event's price or venue after bookings have already occurred, referencing event details dynamically from the ticket document would change the details retroactively for previous buyers. Eventify solves this by denormalizing transaction-critical data directly into a static ticket subdocument.

3. Project Explanation in STAR Methodology

SITUATION

Campus organizers managed event registrations and ticket distribution using fragmented platforms and manual paper lists. This process resulted in slow check-ins at entry gates, ticket overselling due to concurrent access, lack of session security, and database inconsistency if event details shifted mid-campaign.

TASK

My objective was to engineer the complete backend infrastructure, database schemas, and verification testing suites for a secure, concurrency-safe campus event management platform. Key challenges included establishing secure stateless session validations, preventing database race conditions during concurrent bookings, handling file upload streams, and writing integration test coverage to verify API endpoints.

ACTION

- **Secure Auth:** I built a registration and login pipeline using *Bcryptjs* for salting and hashing. I configured stateless session tracking via *JSON Web Tokens (JWT)* stored in secure, *httpOnly* cookies to protect sessions from XSS attacks.
- **Concurrency:** I implemented a concurrency-safe booking logic using MongoDB's atomic *\$inc* update operator. This ensures ticket inventory checks and decrements occur in a single, database-level operation to prevent overselling.
- **Data Modeling:** I designed a denormalized database schema where transactional ticket records capture a snapshot of the event details (price, venue, venue date) at booking time, isolating past purchases from subsequent event updates.
- **File Uploads:** I integrated *Multer* middleware to handle incoming multipart image streams, validating size constraints (5MB max) and routing paths to static directories.
- **Integration Testing:** I wrote 100% of the backend integration test suites using *Jest* and *Supertest*, mocking database transactions in an in-memory Mongo environment to prevent code regressions.

RESULT

I successfully delivered a production-ready, highly secure backend API. The test suite achieved complete coverage of core workflows, verifying register/login scenarios, ticket inventory limits, and access controls. By using atomic operations, we mathematically eliminated ticket double-booking, and HTTP-only JWTs protected user sessions.

4. Detailed Feature Implementation

This section reviews the technical design of Eventify's core backend features, focusing on how different modules communicate and manage data without relying on specific folder layouts:

• User Authentication and Session Management

During user registration, the system validates that all fields are present and that the email does not already exist. The plain-text password is encrypted via Bcrypt's salting algorithm with 10 salt rounds and stored. When logging in, the server matches the email and compares the hash. Upon a match, a JWT is signed with the user's unique ID and stored in a secure cookie. The cookie is configured with httpOnly (denying access to JavaScript), Secure (only sent over HTTPS), and SameSite=Strict (mitigating Cross-Site Request Forgery). A logout request clears this cookie, terminating the session.

• Concurrency-Safe Booking Pipeline

To purchase a ticket, an authenticated user sends a request specifying the event ID and the quantity. The server first reads the event. If the event exists and has sufficient available tickets, the server initiates the creation of a Ticket document. Critically, to update the event's ticket inventory, the backend does not perform a simple write of the pre-calculated value. Instead, it performs an atomic decrement on `availableTickets` and an increment on `currentParticipants` by the purchased quantity. This avoids dirty writes where two threads overwrite each other's changes.

• Transaction Snapshots via Schema Denormalization

Instead of storing just references to the user and event IDs, the Ticket schema utilizes a denormalized subdocument. At the moment of purchase, the event's name, date, time, price, and location are copied into the ticket's `ticketDetails` field. This snapshot preserves historical transactional data. For instance, if the event organizer subsequently changes the ticket price from Rs. 200 to Rs. 500, previous tickets still correctly display Rs. 200, which serves as a financial audit trail and improves query speeds during user dashboard retrieval.

• File Upload and Static Storage Pipeline

Event creation requires organizers to upload promotional banners. Because JSON-based REST APIs are designed for textual payload exchanges, binary uploads must be handled as multipart form data. The server incorporates a file stream parser to intercept incoming streams, validates file size boundaries (5MB maximum) and MIME types (restricting uploads to images), generates a unique filename based on an epoch timestamp, writes it to a designated disk directory, and stores the relative filesystem path in the Event document.

• Integration Test Automation

The backend API incorporates automated integration testing. The test environment utilizes a pre-configured database runner that spins up an in-memory database server. The test files run in isolation: they establish connections, seed mock user data, authenticate via mocked cookies, send HTTP requests to Express endpoints using Supertest, and execute assertions on the responses (such as validating status codes like 201 Created or 409 Conflict, and inspecting body schemas). After each test, collections are purged to ensure zero cross-test interference.

5. Algorithms and Critical System Concepts

For a technical consultant role, understanding the low-level mechanics of core algorithms and design patterns is essential. Here are the core patterns implemented in the backend of Eventify:

• Bcrypt Adaptive Hashing Algorithm

Bcrypt is a blowfish-cipher-based hashing algorithm designed specifically to protect passwords. Unlike standard cryptographic hashes (like SHA-256) which are designed to be fast, Bcrypt is intentionally slow. It incorporates two major safety mechanisms:

1. **Salting:** A random string (salt) is appended to each password before hashing. This results in completely different hashes even if two users have the identical password, neutralizing *rainbow table attacks* (pre-computed hash lists).
2. **Key Stretching & Work Factor:** Bcrypt uses a configurable 'work factor' (rounds=10 in our implementation). The hashing function runs recursively 2^{10} times. This delay has negligible impact on a single login request but makes brute-force cracking on GPU clusters computationally expensive and impractical.

• Database-Level Atomic Increment/Decrement

In multi-user database operations, race conditions occur when a process reads data, changes it locally, and writes it back, while another process does the same. If two bookings read 5 available tickets at the same instant and both write back $5 - 1 = 4$, the DB will record 4 available tickets instead of 3, causing a double-allocation error.

To solve this, our backend uses MongoDB's atomic operator **\$inc**: ``Event.findByIdAndUpdate(eventId, { $inc: { availableTickets: -quantity } })``. This ensures that reading the current value, subtracting, and writing it back is compiled into a single, indivisible transaction execution at the database engine level, blocking intermediate reads and eliminating race conditions.

6. Key Terminology Glossary

These definitions reflect how these terms apply directly to your project's codebase, providing clear answers for conceptual interview questions:

JWT (JSON Web Token): A compact, URL-safe means of representing claims to be transferred between two parties. Composed of a Header, Payload, and Signature. It is cryptographically signed, allowing the backend to trust its authenticity without looking it up in the database on every request.

httpOnly Cookie: An HTTP cookie header configuration that prevents client-side APIs (like JavaScript's document.cookie) from accessing the cookie. Storing session JWTs in httpOnly cookies protects them from Cross-Site Scripting (XSS) token-stealing attacks.

Bcrypt: An adaptive, blowfish-based password-hashing function that incorporates salting and recursive key stretching to slow down execution times, protecting passwords against offline dictionary and brute-force attacks.

Middleware: A software handler function in Express that has access to the Request and Response objects, and the next middleware function in the application's request-response cycle. Used in our app for authentication checks and file uploads.

REST API: Representational State Transfer. An architectural style for designing networked applications. It utilizes standard HTTP methods (GET, POST, PUT, DELETE) to represent CRUD operations on resource URIs (e.g. /events/:id).

CRUD: Create, Read, Update, Delete. The four basic operations of persistent storage. Our API maps these to POST /createEvent, GET /events, PUT /events/:id, and DELETE /events/:id.

ODM (Object Document Mapper): An abstraction layer that maps database documents (NoSQL) to JavaScript object representations, providing schema validation, type casting, and relationship population (like Mongoose for MongoDB).

Race Condition: A concurrency bug where the output of a process is dependent on the sequence or timing of uncontrollable events (like multiple requests trying to modify the same resource simultaneously). Resolved via atomic updates.

Denormalization: A database design strategy of intentionally storing redundant copies of data in multiple collections to optimize read performance and preserve historical snapshots of transactional data (like booking price).

Integration Testing: A testing level where individual software modules are combined and tested as a group to verify that endpoints, routes, databases, and authentication layers interact correctly under simulated scenarios.

Node.js Event Loop: The mechanism that allows Node.js to perform non-blocking I/O operations despite JavaScript being single-threaded, offloading operations to the system kernel or thread pool and executing callbacks when ready.

7. Architectural Tech Stack Rationale

For a technical consultant, understanding the 'Why' behind technology choices is more critical than the 'How'. Here is the architectural analysis of our stack choices compared to alternatives:

Why the MERN Stack?

- **Unified Language Ecosystem:** Using JavaScript/JSX on both the frontend (React) and backend (Node.js) streamlines development. Developers can share utility functions, data validations, and state models across boundaries, increasing development speed.
- **JSON Native Flow:** Frontend state, REST API payloads, and database documents (MongoDB BSON) are all represented as JSON objects. This eliminates translation layers (like mapping SQL tables to JSON arrays), reducing serialization overhead.
- **High Concurrency Handling:** Node.js is optimized for asynchronous, input-output heavy workloads, making it perfect for real-time portals that process multiple concurrent API requests.

Selected Tech (MERN / Node.js)	Alternative (Python Django / Relational SQL)
Express & Node.js: Lightweight, asynchronous, micro-framework structure. Offers raw speed, event-driven scaling, and granular control over middleware.	Django (Python): Opinions-heavy, monolithic structure. While it offers built-in admin interfaces, its synchronous nature can introduce high performance overhead under concurrent API traffic.
MongoDB: Schema-less NoSQL document model. Rapid development cycle, nested documents map perfectly to dynamic UI state (e.g. event comments lists).	SQL Databases (PostgreSQL): Rigid schema layouts require database migrations for every modification. Columns must be pre-defined, making rapid prototyping and changing requirements slower.

8. Database Model Evaluation

A key design decision in Eventify was using a NoSQL Document Model (MongoDB/Mongoose) rather than a Relational SQL Database. Below is a detailed breakdown of the trade-offs:

Document Model (MongoDB NoSQL)	Relational Model (PostgreSQL / MySQL)
Flexible Schema: Event parameters can evolve dynamically without schema migrations (e.g., adding field arrays like 'likes' or 'comments' nested directly in the event).	Rigid Schema: Every schema shift requires table migrations. Columns must be pre-defined, making rapid prototyping and changing requirements slower.
High Read Performance: Related data is nested (embedded documents). Querying an event fetches its comments and owner references in a single database read without JOIN overhead.	Join Performance Cost: Accessing nested relations requires querying multiple tables using foreign key indexes and joining them, which can degrade performance on complex tables.

Denormalization Pattern: Simple replication of transactional data. Resolves pricing shifts by snapshotting values (e.g. price) inside the ticket subdocument.	Strict Normalization (3NF): Reduces data redundancy by isolating values in separate tables. Preserving snapshots requires duplicate mapping columns, bypassing normalization rules.
--	--

Advantages and Disadvantages of the MongoDB Document Model

Advantages:

- 1. **Natural Mapping:** Documents map directly to JavaScript objects, eliminating the object-relational impedance mismatch.
- 2. **Horizontal Scaling:** MongoDB scales horizontally using sharding, dividing datasets across servers, whereas SQL scale-up requires vertical server upgrades.
- 3. **Embedded Documents:** Storing comment arrays directly inside the Event document provides faster read speeds by avoiding database joins.

Disadvantages:

- 1. **Lack of Strict ACID Transactions:** Unlike SQL databases which guarantee ACID compliance across multiple tables natively, MongoDB transactions across multiple documents are more expensive and complex to implement.
- 2. **Data Duplication (Denormalization):** Storing event snapshots inside the Ticket collection increases database storage requirements.
- 3. **Data Inconsistency Risks:** Since MongoDB does not enforce relational constraints (like ON DELETE CASCADE), managing orphaned documents (e.g., deleting tickets when an event is deleted) must be handled at the application logic layer.